# CS5494 Assignment 1 - Task 1.2

## Neural Autoregressive Density Estimator (NADE) on binarized MNIST

**Name:** Zhu Pengyu **EID:** pengyuzhu3

This notebook implements the neural autoregressive model required by Task 1.2. It follows Lecture 3 and uses the shared `data.py` supplied with Task 1.1 so that NADE and FVSBN use exactly the same MNIST split, binarization rule and row-major pixel order.

Run order:

1. Keep `QUICK_RUN=True` and run all model/training cells once as a smoke test.
2. Set `QUICK_RUN=False`, restart the kernel, and run all cells for the final run.
3. Set `RUN_ABLATION=True` only when the main run is working.

The notebook writes all generated artifacts under `build/`.

# 1. Imports and reproducibility

Only packages already used by Task 1.1 are required: PyTorch, torchvision, NumPy and Matplotlib.

In [1]:
```
import csv
import json
import math
import os
import random
import time

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset

import data as D

SEED = 0
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Python/PyTorch environment ready")
```

```
print("PyTorch:", torch.__version__)
print("Device:", device)
if device.type == "cuda":
    print("GPU:", torch.cuda.get_device_name(0))

BUILD = "build"
RUNS = os.path.join(BUILD, "runs")
FIGS = os.path.join(BUILD, "figs")
os.makedirs(RUNS, exist_ok=True)
os.makedirs(FIGS, exist_ok=True)
```

```
Python/PyTorch environment ready
PyTorch: 2.14.0+cu130
Device: cuda
GPU: NVIDIA GeForce RTX 3060 Laptop GPU
```

# 2. Experiment configuration

`QUICK_RUN=True` is only for checking that the implementation works. It is **not** the final assignment experiment. For the final report, set it to `False`, restart the kernel and run the notebook again.

In [2]:
```
QUICK_RUN = False
RUN_ABLATION = False

if QUICK_RUN:
    N_TRAIN = 5000
    N_TEST = 2000
    EPOCHS = 3
    STAGE_EPOCHS = (1, 2, 3)
else:
    N_TRAIN = 60000
    N_TEST = 10000
    EPOCHS = 50
    STAGE_EPOCHS = (1, 5, 10, 25, 50)

DIM = 784
HIDDEN = 128
BATCH_SIZE = 64
LEARNING_RATE = 1e-3
THRESHOLD = 0.5
PIXEL_ORDER = "row"

print({
    "quick_run": QUICK_RUN,
    "n_train": N_TRAIN,
    "n_test": N_TEST,
    "epochs": EPOCHS,
    "hidden": HIDDEN,
    "batch_size": BATCH_SIZE,
    "learning_rate": LEARNING_RATE,
    "threshold": THRESHOLD,
    "pixel_order": PIXEL_ORDER,
    "seed": SEED,
})
```

```
{'quick_run': False, 'n_train': 60000, 'n_test': 10000, 'epochs': 50, 'hidden': 1
28, 'batch_size': 64, 'learning_rate': 0.001, 'threshold': 0.5, 'pixel_order': 'r
ow', 'seed': 0}
```

# 3. Shared MNIST preprocessing

Task 1.1 and Task 1.2 must use the same data contract:

- deterministic binarization with threshold 0.5;
- values exactly in `{0, 1}`;
- flattened row-major to 784 pixels;
- fixed subset seed 0 when a subset is used.

In [3]:
```
x_train_all, y_train_all = D.load_binarized(
    "train", threshold=THRESHOLD, mode="threshold", seed=SEED
)
x_test_all, y_test_all = D.load_binarized(
    "test", threshold=THRESHOLD, mode="threshold", seed=SEED
)

x_train, y_train = D.take_subset(
    x_train_all, y_train_all, N_TRAIN, seed=SEED
)
x_test, y_test = D.take_subset(
    x_test_all, y_test_all, N_TEST, seed=SEED
)

perm = D.pixel_order(PIXEL_ORDER)
x_train = D.apply_order(x_train, perm)
x_test = D.apply_order(x_test, perm)

assert x_train.dtype == np.float32
assert x_train.shape[1] == DIM
assert set(np.unique(x_train)).issubset({0.0, 1.0})

print("train:", x_train.shape, "test:", x_test.shape)
print("train ink fraction:", float(x_train.mean()))
print("test ink fraction:", float(x_test.mean()))

preview = D.undo_order(x_train[:16], perm)
D.show_grid(preview, rows=2, cols=8, title="Shared binarized MNIST training samp
plt.show()
```

```
  0%|          | 0.00/9.91M [00:00<?, ?B/s]

  0%|          | 32.8k/9.91M [00:00<01:07, 147kB/s]

  1%|          | 98.3k/9.91M [00:00<00:42, 229kB/s]

  2%|▏         | 164k/9.91M [00:00<00:38, 254kB/s]

  3%|▎         | 295k/9.91M [00:00<00:25, 379kB/s]

  5%|▍         | 459k/9.91M [00:01<00:18, 500kB/s]
```

```
  9%|▉          | 852k/9.91M [00:01<00:09, 921kB/s]
 17%|█▋         | 1.64M/9.91M [00:01<00:04, 1.75MB/s]
 26%|██▋        | 2.56M/9.91M [00:01<00:02, 3.01MB/s]
 33%|███▍       | 3.28M/9.91M [00:01<00:01, 3.64MB/s]
 42%|████▎      | 4.19M/9.91M [00:01<00:01, 4.78MB/s]
 52%|█████▏     | 5.11M/9.91M [00:02<00:00, 5.77MB/s]
 59%|█████▉     | 5.83M/9.91M [00:02<00:01, 3.66MB/s]
 84%|████████▍  | 8.32M/9.91M [00:02<00:00, 6.00MB/s]
100%|██████████| 9.91M/9.91M [00:02<00:00, 3.72MB/s]

  0%|          | 0.00/28.9k [00:00<?, ?B/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 103kB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 103kB/s]

  0%|          | 0.00/1.65M [00:00<?, ?B/s]
  2%|▏         | 32.8k/1.65M [00:00<00:12, 129kB/s]
  6%|▌         | 98.3k/1.65M [00:00<00:07, 221kB/s]
 12%|█▏        | 197k/1.65M [00:01<00:07, 197kB/s]
 20%|█▉        | 328k/1.65M [00:01<00:04, 302kB/s]
 34%|███▍      | 557k/1.65M [00:01<00:02, 491kB/s]
 60%|█████▉    | 983k/1.65M [00:01<00:00, 876kB/s]
 83%|████████▎ | 1.38M/1.65M [00:01<00:00, 1.13MB/s]
 91%|█████████ | 1.51M/1.65M [00:02<00:00, 775kB/s]
100%|██████████| 1.65M/1.65M [00:02<00:00, 691kB/s]

  0%|          | 0.00/4.54k [00:00<?, ?B/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 4.54MB/s]
```

```
train: (60000, 784) test: (10000, 784)
train ink fraction: 0.1322583109140396
test ink fraction: 0.13422946631908417
```

```
C:\Users\zpy\AppData\Local\Temp\ipykernel_4120\493140892.py:29: UserWarning: Figu
reCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

# 4. NADE model

For a binary image $x = (x_1, \ldots, x_D)$, NADE models

$$p(x) = \prod_{i=1}^{D} p(x_i \mid x_{<i}).$$

A shared hidden layer represents the prefix:

$$a_i = c + \sum_{j<i} W_{:,j} x_j, \qquad h_i = \sigma(a_i),$$

and the current Bernoulli probability is

$$p(x_i = 1 \mid x_{<i}) = \sigma(V_i^T h_i + b_i).$$

The shifted cumulative sum in `logits` guarantees that output $i$ never sees the current or future pixels. Training is parallel over all target pixels; sampling remains sequential because each newly sampled pixel becomes part of the next prefix.

In [4]:
```python
class NADE(nn.Module):
    def __init__(self, dim=784, hidden=128):
        super().__init__()
        self.dim = dim
        self.hidden = hidden

        # Shared input-to-hidden weights and hidden-to-output weights.
        self.W = nn.Parameter(torch.empty(hidden, dim))
        self.V = nn.Parameter(torch.empty(dim, hidden))
        self.c = nn.Parameter(torch.zeros(hidden))
        self.b = nn.Parameter(torch.zeros(dim))

        nn.init.xavier_uniform_(self.W)
        nn.init.xavier_uniform_(self.V)

    def logits(self, x):
        # contribution[b, h, i] = W[h, i] * x[b, i]
        contribution = x.unsqueeze(1) * self.W.unsqueeze(0)
        prefix = torch.cumsum(contribution, dim=2)

        zero_prefix = torch.zeros(
            x.size(0), self.hidden, 1,
            device=x.device, dtype=x.dtype
        )

        # Shift by one position: prediction i only receives x_<i.
        hidden_input = self.c.view(1, -1, 1) + torch.cat(
            [zero_prefix, prefix[:, :, :-1]], dim=2
        )
        hidden_state = torch.sigmoid(hidden_input)
```

```
        return torch.einsum("bhd,dh->bd", hidden_state, self.V) + self.b

    def nll(self, x):
        return F.binary_cross_entropy_with_logits(
            self.logits(x), x, reduction="none"
        ).sum(dim=1)

    @torch.no_grad()
    def sample(self, n, device=None):
        device = device or self.b.device
        samples = torch.zeros(n, self.dim, device=device)
        hidden_input = self.c.unsqueeze(0).expand(n, -1).clone()

        for i in range(self.dim):
            hidden_state = torch.sigmoid(hidden_input)
            logit = (
                hidden_state * self.V[i].unsqueeze(0)
            ).sum(dim=1) + self.b[i]
            probability = torch.sigmoid(logit)
            current_pixel = torch.bernoulli(probability)
            samples[:, i] = current_pixel
            hidden_input = hidden_input + (
                current_pixel.unsqueeze(1) * self.W[:, i].unsqueeze(0)
            )

        return samples


model_check = NADE(dim=8, hidden=4)
toy_x = torch.bernoulli(torch.full((5, 8), 0.3))
assert model_check.logits(toy_x).shape == toy_x.shape
assert model_check.nll(toy_x).shape == (5,)
assert model_check.sample(3).shape == (3, 8)
print("NADE shape and sampling checks passed")
```

```
NADE shape and sampling checks passed
```

# 5. Evaluation, timing and training helpers

NLL is summed over all 784 pixels and averaged over images. Bits per dimension is

$$\text{bpd} = \frac{\text{NLL}}{784 \log 2}.$$

Sampling time follows the Task 1.1 protocol: one warm-up call, five timed repetitions, CUDA synchronization and the median time for 100 samples.

In [5]:
```
def parameter_count(model):
    return int(sum(p.numel() for p in model.parameters() if p.requires_grad))


def evaluate_nll(model, x, batch_size=128):
    model.eval()
    loader = DataLoader(
        TensorDataset(torch.from_numpy(np.ascontiguousarray(x)).float()),
        batch_size=batch_size,
        shuffle=False,
```

```
    )
    total = 0.0
    count = 0
    with torch.no_grad():
        for (batch_x,) in loader:
            batch_x = batch_x.to(device)
            total += model.nll(batch_x).sum().item()
            count += batch_x.size(0)
    return total / count


def time_sampling(model, n=100, repeats=5):
    model.eval()
    model.sample(n, device=device)
    if device.type == "cuda":
        torch.cuda.synchronize()

    measurements = []
    for _ in range(repeats):
        if device.type == "cuda":
            torch.cuda.synchronize()
        start = time.perf_counter()
        model.sample(n, device=device)
        if device.type == "cuda":
            torch.cuda.synchronize()
        measurements.append(time.perf_counter() - start)
    return float(np.median(measurements))


def train_nade(
    x_train,
    x_eval,
    hidden=128,
    epochs=50,
    batch_size=64,
    learning_rate=1e-3,
    stage_epochs=(1, 5, 10, 25, 50),
    stage_samples=4,
    seed=0,
    verbose=True,
):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

    model = NADE(dim=x_train.shape[1], hidden=hidden).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    train_tensor = torch.from_numpy(np.ascontiguousarray(x_train)).float()
    generator = torch.Generator().manual_seed(seed)
    loader = DataLoader(
        TensorDataset(train_tensor),
        batch_size=batch_size,
        shuffle=True,
        generator=generator,
    )

    history = {
```

```
        "epoch": [],
        "train_nll": [],
        "eval_nll": [],
        "eval_bpd": [],
    }
    stages = {}

    if device.type == "cuda":
        torch.cuda.synchronize()
    training_start = time.perf_counter()

    for epoch in range(1, epochs + 1):
        model.train()
        total = 0.0
        count = 0

        for (batch_x,) in loader:
            batch_x = batch_x.to(device)
            loss = model.nll(batch_x).mean()

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total += loss.item() * batch_x.size(0)
            count += batch_x.size(0)

        train_nll = total / count
        eval_nll = evaluate_nll(model, x_eval, batch_size=batch_size)
        eval_bpd = eval_nll / (x_train.shape[1] * math.log(2.0))

        history["epoch"].append(epoch)
        history["train_nll"].append(train_nll)
        history["eval_nll"].append(eval_nll)
        history["eval_bpd"].append(eval_bpd)

        if epoch in stage_epochs:
            stages[epoch] = model.sample(
                stage_samples, device=device
            ).cpu().numpy()

        if verbose and (epoch == 1 or epoch % 5 == 0 or epoch == epochs):
            print(
                f"epoch {epoch:3d} | train NLL {train_nll:8.2f} | "
                f"eval NLL {eval_nll:8.2f} | eval bpd {eval_bpd:.4f}"
            )

    if device.type == "cuda":
        torch.cuda.synchronize()
    training_time = time.perf_counter() - training_start

    sample_time_100 = time_sampling(model, n=100, repeats=5)
    final_samples = model.sample(40, device=device).cpu().numpy()

    return {
        "model": model,
        "history": history,
        "stages": stages,
        "final_samples": final_samples,
        "training_time": float(training_time),
```

```
        "sample_time_100": sample_time_100,
        "parameters": parameter_count(model),
    }
```

# 6. Main experiment

The quick run verifies the complete pipeline. The final numbers used in the report must come from `QUICK_RUN=False`.

In [6]:
```
result = train_nade(
    x_train=x_train,
    x_eval=x_test,
    hidden=HIDDEN,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    learning_rate=LEARNING_RATE,
    stage_epochs=STAGE_EPOCHS,
    stage_samples=4,
    seed=SEED,
)

model = result["model"]
history = result["history"]

final_nll = history["eval_nll"][-1]
final_bpd = history["eval_bpd"][-1]

print("\nFinal experiment summary")
print("quick run:", QUICK_RUN)
print("test/eval NLL per image:", final_nll)
print("test/eval bits per dimension:", final_bpd)
print("parameters:", result["parameters"])
print("training time (s):", result["training_time"])
print("time for 100 samples (s):", result["sample_time_100"])
```

```
epoch   1 | train NLL   170.81 | eval NLL   128.83 | eval bpd 0.2371
epoch   5 | train NLL    86.75 | eval NLL    84.24 | eval bpd 0.1550
epoch  10 | train NLL    77.30 | eval NLL    76.77 | eval bpd 0.1413
epoch  15 | train NLL    74.11 | eval NLL    74.20 | eval bpd 0.1365
epoch  20 | train NLL    72.42 | eval NLL    72.90 | eval bpd 0.1342
epoch  25 | train NLL    71.34 | eval NLL    72.09 | eval bpd 0.1327
epoch  30 | train NLL    70.58 | eval NLL    71.49 | eval bpd 0.1316
epoch  35 | train NLL    70.00 | eval NLL    71.10 | eval bpd 0.1308
epoch  40 | train NLL    69.55 | eval NLL    70.77 | eval bpd 0.1302
epoch  45 | train NLL    69.17 | eval NLL    70.52 | eval bpd 0.1298
epoch  50 | train NLL    68.86 | eval NLL    70.28 | eval bpd 0.1293
Final experiment summary
quick run: False
test/eval NLL per image: 70.28168876953124
test/eval bits per dimension: 0.12933041307793913
parameters: 201616
training time (s): 244.13101360000292
time for 100 samples (s): 0.10700630000064848
```

# 7. Save the run and generate required figures

In [7]:
```
run_name = "nade_quick" if QUICK_RUN else "nade_main_60k_ep50"

run_payload = {
    "name": run_name,
    "model": "NADE",
    "quick_run": QUICK_RUN,
    "n_train": int(len(x_train)),
    "n_test": int(len(x_test)),
    "epochs": EPOCHS,
    "batch_size": BATCH_SIZE,
    "learning_rate": LEARNING_RATE,
    "hidden": HIDDEN,
    "threshold": THRESHOLD,
    "pixel_order": PIXEL_ORDER,
    "seed": SEED,
    "device": str(device),
    "gpu": torch.cuda.get_device_name(0) if device.type == "cuda" else "cpu",
    "parameters": result["parameters"],
    "training_time_s": result["training_time"],
    "sample_time_100_s": result["sample_time_100"],
    "final_nll": final_nll,
    "final_bpd": final_bpd,
    "history": history,
}

with open(os.path.join(RUNS, run_name + ".json"), "w", encoding="utf-8") as file
    json.dump(run_payload, file, indent=2)

np.savez_compressed(
    os.path.join(RUNS, run_name + ".npz"),
    final=result["final_samples"],
    perm=perm,
    **{f"stage_{epoch}": samples for epoch, samples in result["stages"].items()}
)

torch.save(model.state_dict(), os.path.join(RUNS, run_name + ".pt"))
print("Saved run:", run_name)
```

```
Saved run: nade_main_60k_ep50
```

In [8]:
```
fig, ax1 = plt.subplots(figsize=(7, 4))
ax1.plot(history["epoch"], history["train_nll"], label="train NLL")
ax1.plot(history["epoch"], history["eval_nll"], label="eval NLL")
ax1.set_xlabel("epoch")
ax1.set_ylabel("NLL per image")
ax1.grid(alpha=0.25)
ax1.legend(loc="upper right")

ax2 = ax1.twinx()
ax2.plot(history["epoch"], history["eval_bpd"], "k--", label="eval bpd")
ax2.set_ylabel("bits per dimension")

plt.title("NADE training and evaluation loss")
fig.tight_layout()
loss_path = os.path.join(FIGS, "fig_nade_loss.png")
fig.savefig(loss_path, dpi=180, bbox_inches="tight")
plt.show()
print(loss_path)
```

```
build\figs\fig_nade_loss.png
```

```
C:\Users\zpy\AppData\Local\Temp\ipykernel_4120\4282775652.py:17: UserWarning: Fig
ureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

In [9]:
```
stage_epochs_available = sorted(result["stages"])
stage_images = np.concatenate(
    [D.undo_order(result["stages"][epoch], perm) for epoch in stage_epochs_avail
    axis=0,
)
stage_titles = [f"epoch {epoch}" for epoch in stage_epochs_available]
stage_path = os.path.join(FIGS, "fig_nade_stages.png")
D.show_grid(
    stage_images,
    rows=len(stage_epochs_available),
    cols=4,
    path=stage_path,
    titles=stage_titles,
    title="NADE samples during training",
)
plt.figure(figsize=(10, 2.2 * len(stage_epochs_available)))
plt.imshow(plt.imread(stage_path))
plt.axis("off")
plt.show()

final_images = D.undo_order(result["final_samples"], perm)
final_path = os.path.join(FIGS, "fig_nade_samples.png")
D.show_grid(
    final_images,
    rows=5,
    cols=8,
    path=final_path,
    title="Final NADE samples",
)
plt.figure(figsize=(10, 7))
plt.imshow(plt.imread(final_path))
plt.axis("off")
plt.show()

print(stage_path)
print(final_path)
```

```
C:\Users\zpy\AppData\Local\Temp\ipykernel_4120\1778453934.py:19: UserWarning: Fig
ureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
build\figs\fig_nade_stages.png
build\figs\fig_nade_samples.png
C:\Users\zpy\AppData\Local\Temp\ipykernel_4120\1778453934.py:33: UserWarning: Fig
ureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

## 8. Teacher-forced conditional predictions

This figure separates conditional prediction quality from free-running ancestral sampling. During teacher forcing, each prediction receives the true prefix; during sampling, it receives the model's earlier samples, so errors may accumulate.

In [10]:
```
model.eval()
example_tensor = torch.from_numpy(x_test[:6]).float().to(device)
with torch.no_grad():
    probabilities = torch.sigmoid(model.logits(example_tensor)).cpu().numpy()

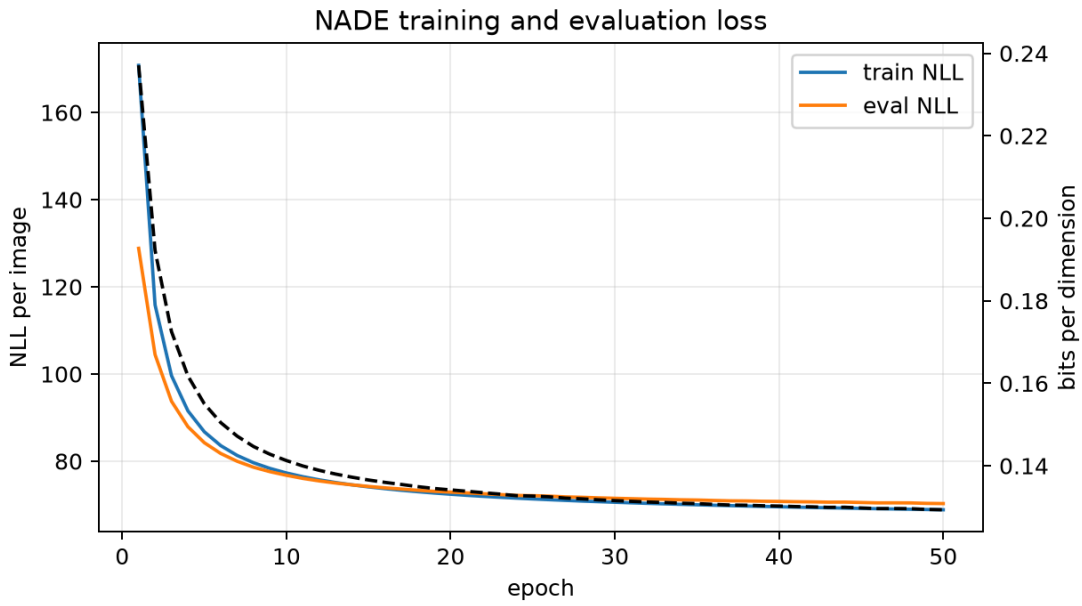
original = D.undo_order(x_test[:6], perm).reshape(-1, 28, 28)
probability_maps = D.undo_order(probabilities, perm).reshape(-1, 28, 28)
argmax_maps = (probability_maps >= 0.5).astype(np.float32)

fig, axes = plt.subplots(6, 3, figsize=(7, 12))
for row in range(6):
    axes[row, 0].imshow(original[row], cmap="gray", vmin=0, vmax=1)
    axes[row, 1].imshow(probability_maps[row], cmap="gray", vmin=0, vmax=1)
    axes[row, 2].imshow(argmax_maps[row], cmap="gray", vmin=0, vmax=1)
    for col in range(3):
        axes[row, col].axis("off")
axes[0, 0].set_title("test image")
axes[0, 1].set_title("conditional probability")
axes[0, 2].set_title("thresholded prediction")
fig.tight_layout()
pred_path = os.path.join(FIGS, "fig_nade_predmap.png")
fig.savefig(pred_path, dpi=180, bbox_inches="tight")
plt.show()
print(pred_path)
```

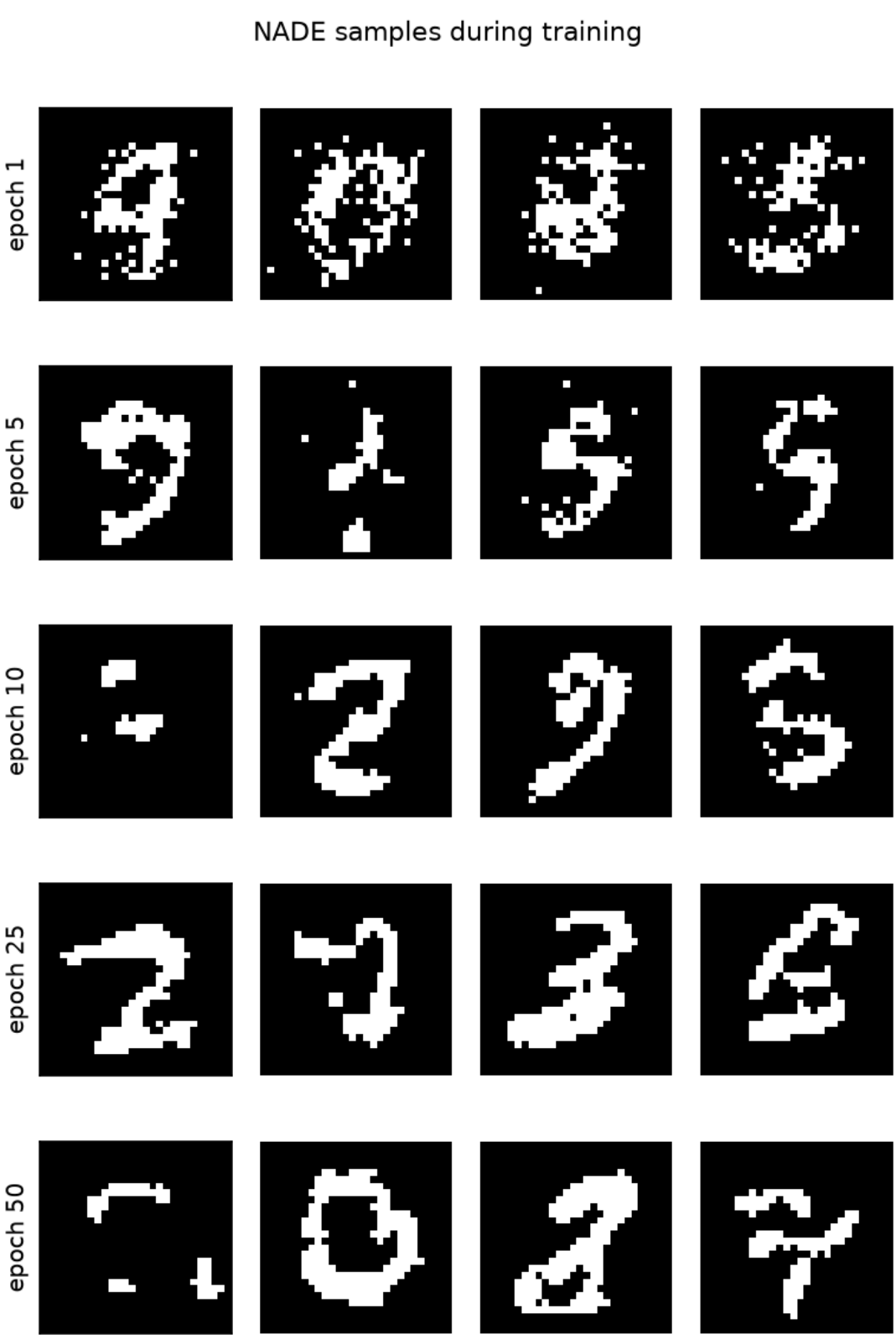
```
build\figs\fig_nade_predmap.png
```

```
C:\Users\zpy\AppData\Local\Temp\ipykernel_4120\1791546170.py:23: UserWarning: Fig
ureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

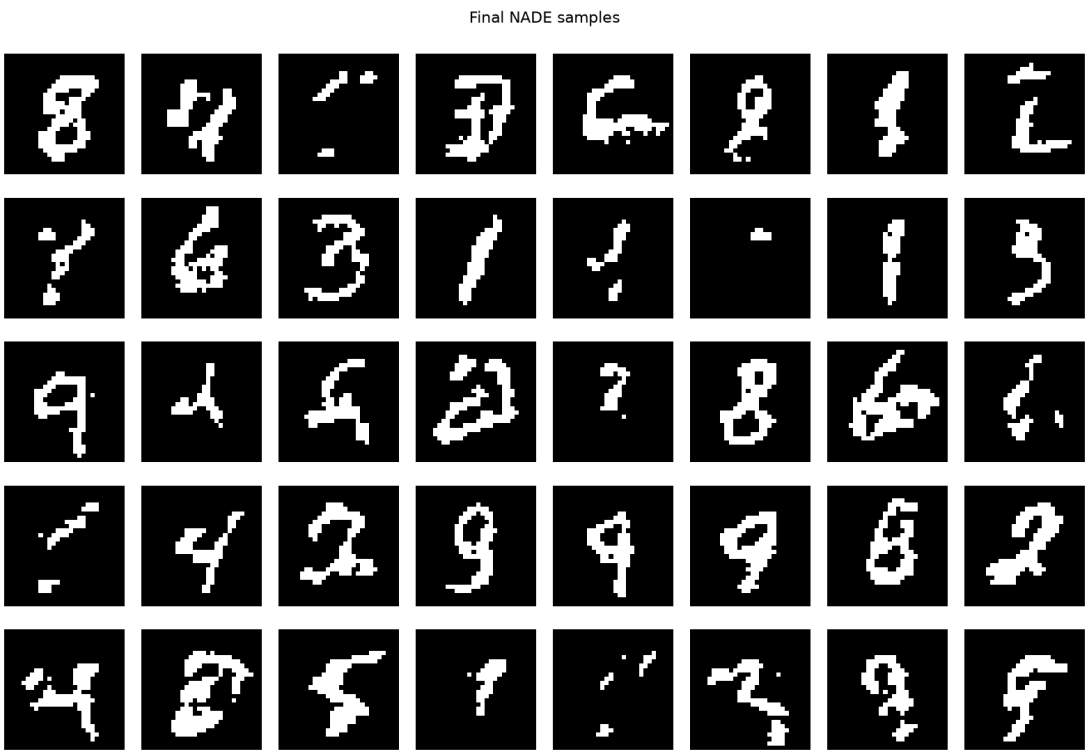
## Loss curve



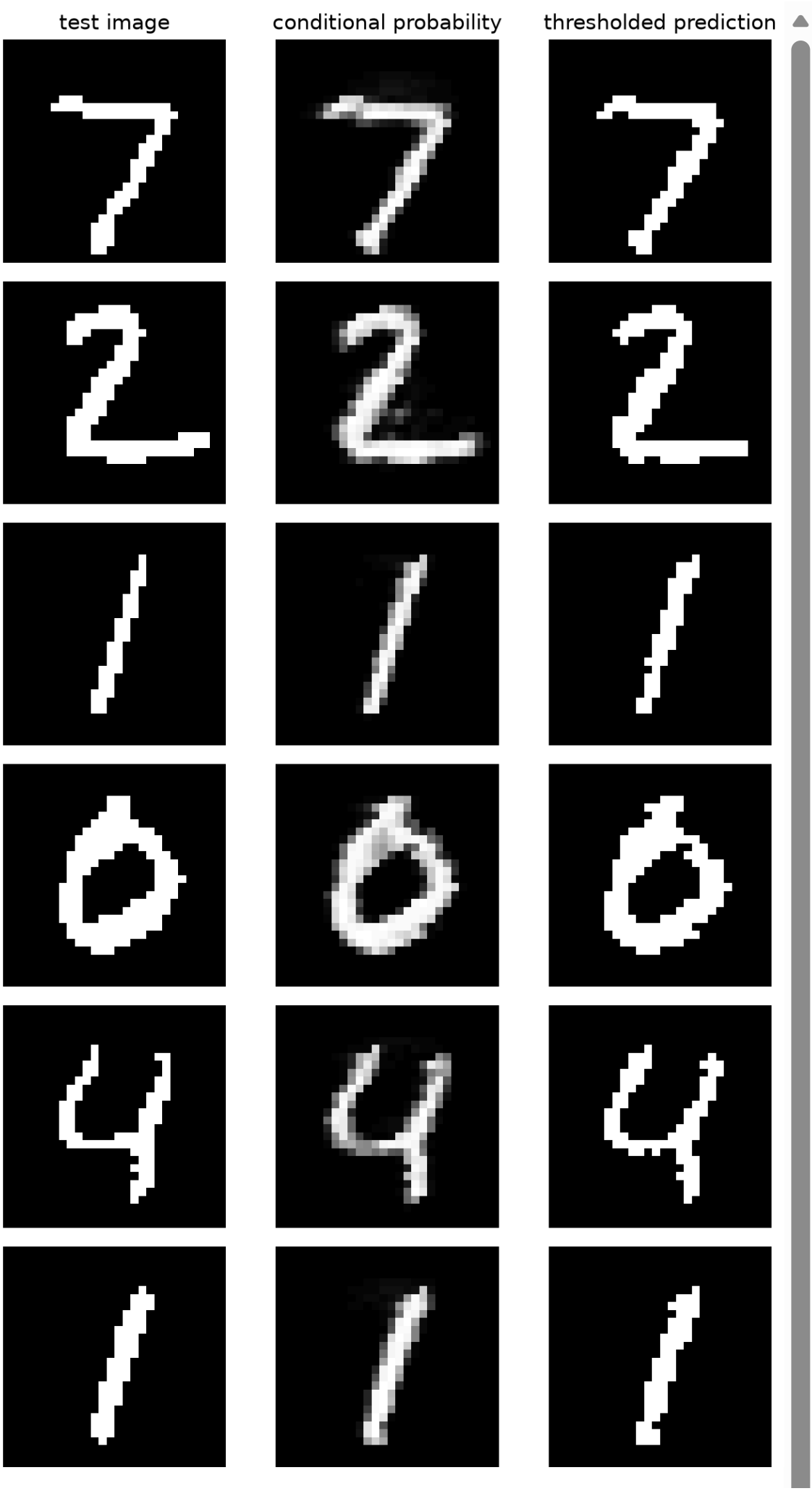

## Samples during training




Final samples




Teacher-forced conditional predictions


test image
conditional probability
thresholded prediction

# 9. Hidden-width ablation

This is intentionally disabled for the first run. When the main model works, set `RUN_ABLATION=True`. The final ablation uses 20,000 training images, 30 epochs and hidden widths 64, 128 and 256 while keeping all other settings fixed.

In [11]:
```
ablation_results = []

if RUN_ABLATION:
    if QUICK_RUN:
        ablation_n_train = 2000
        ablation_n_test = 1000
        ablation_epochs = 2
    else:
        ablation_n_train = 20000
        ablation_n_test = 10000
        ablation_epochs = 30

    x_ablation, y_ablation = D.take_subset(
        x_train_all, y_train_all, ablation_n_train, seed=SEED
    )
    x_ablation_test, _ = D.take_subset(
        x_test_all, y_test_all, ablation_n_test, seed=SEED
    )
    x_ablation = D.apply_order(x_ablation, perm)
    x_ablation_test = D.apply_order(x_ablation_test, perm)

    for hidden_size in (64, 128, 256):
        print("\nHidden size:", hidden_size)
        current = train_nade(
            x_train=x_ablation,
            x_eval=x_ablation_test,
            hidden=hidden_size,
            epochs=ablation_epochs,
            batch_size=BATCH_SIZE,
            learning_rate=LEARNING_RATE,
            stage_epochs=(),
            stage_samples=0,
            seed=SEED,
            verbose=False,
        )

        ablation_results.append({
            "hidden": hidden_size,
            "parameters": current["parameters"],
            "test_nll": current["history"]["eval_nll"][-1],
            "test_bpd": current["history"]["eval_bpd"][-1],
            "training_time_s": current["training_time"],
            "sample_time_100_s": current["sample_time_100"],
        })
        print(ablation_results[-1])

    csv_path = os.path.join(BUILD, "nade_hidden_ablation.csv")
    with open(csv_path, "w", newline="", encoding="utf-8") as file:
        writer = csv.DictWriter(file, fieldnames=ablation_results[0].keys())
        writer.writeheader()
```

```
        writer.writerows(ablation_results)
    print("Saved:", csv_path)
else:
    print("Ablation skipped. Set RUN_ABLATION=True when ready.")
```

```
Ablation skipped. Set RUN_ABLATION=True when ready.
```

In [12]:
```
if ablation_results:
    hidden_sizes = [row["hidden"] for row in ablation_results]
    bpd_values = [row["test_bpd"] for row in ablation_results]

    plt.figure(figsize=(6, 4))
    plt.bar([str(value) for value in hidden_sizes], bpd_values)
    plt.xlabel("hidden units")
    plt.ylabel("test bits per dimension")
    plt.title("NADE hidden-width ablation")
    plt.grid(axis="y", alpha=0.25)
    ablation_path = os.path.join(FIGS, "fig_nade_hidden_ablation.png")
    plt.tight_layout()
    plt.savefig(ablation_path, dpi=180, bbox_inches="tight")
    plt.show()
    print(ablation_path)
```

# 10. Task 1 comparison and handoff files

Only FVSBN and NADE have exact likelihoods on the same binarized data, so their bpd values can be compared directly. GAN losses must not be placed in the same numerical comparison.

In [13]:
```
fvsbn_reference = {
    "test_bpd": 0.1394,
    "test_nll": 75.75,
    "parameters": 615440,
    "training_time_s": "30-50",
    "sample_time_100_s": 0.06,
}

print("FVSBN reference:", fvsbn_reference)
print("NADE current run:", {
    "test_bpd": final_bpd,
    "test_nll": final_nll,
    "parameters": result["parameters"],
    "training_time_s": result["training_time"],
    "sample_time_100_s": result["sample_time_100"],
})

comparison_row = (
    f"| NADE (section 4) | {final_bpd:.4f} | {result['parameters']:,} | "
    f"{result['training_time']:.1f} s | {result['sample_time_100']:.3f} s | "
    "one hidden layer, shared nonlinear conditionals |"
)

report_values = (
    "# Task 1.2 handoff values\n\n"
    f"- Quick run: {QUICK_RUN}\n"
    "- Model: one-hidden-layer NADE\n"
    f"- Hidden units: {HIDDEN}\n"
    f"- Train/test images: {len(x_train)} / {len(x_test)}\n"
```

```
    f"- Epochs: {EPOCHS}\n"
    f"- Test NLL per image: {final_nll:.4f} nats\n"
    f"- Test bits per dimension: {final_bpd:.4f}\n"
    f"- Parameters: {result['parameters']:,}\n"
    f"- Training time: {result['training_time']:.3f} s\n"
    f"- Time for 100 samples: {result['sample_time_100']:.6f} s\n"
    "- Entry point: CS5494_Assignment1_Task2_NADE.ipynb\n\n"
    "## Comparison-table row\n\n"
    f"{comparison_row}\n"
)

handoff_path = os.path.join(BUILD, "task2_handoff_values.md")
with open(handoff_path, "w", encoding="utf-8") as file:
    file.write(report_values)

print(comparison_row)
print("Saved:", handoff_path)
```

```
FVSBN reference: {'test_bpd': 0.1394, 'test_nll': 75.75, 'parameters': 615440, 't
raining_time_s': '30-50', 'sample_time_100_s': 0.06}
NADE current run: {'test_bpd': 0.12933041307793913, 'test_nll': 70.2816887695312
4, 'parameters': 201616, 'training_time_s': 244.13101360000292, 'sample_time_100_
s': 0.10700630000064848}
| NADE (section 4) | 0.1293 | 201,616 | 244.1 s | 0.107 s | one hidden layer, sha
red nonlinear conditionals |
Saved: build\task2_handoff_values.md
```

# 11. Report Section 4: Neural Autoregressive Model

This section summarizes Task 1.2 using the completed 60,000-image, 50-epoch run above. Quick-run values are not used.

## 4.1 Model

The joint distribution of a binarized image is factorized in row-major pixel order as

$$p(\mathbf{x}) = \prod_{i=1}^{784} p(x_i \mid x_{<i}).$$

A one-hidden-layer Neural Autoregressive Density Estimator (NADE) is used to model these conditionals. For pixel $i$, its hidden representation and Bernoulli probability are

$$\mathbf{h}_i = \sigma\left(\mathbf{c} + \sum_{j<i} \mathbf{W}_{:,j} x_j\right), \qquad p(x_i = 1 \mid x_{<i}) = \sigma(\mathbf{V}_{i,:}\mathbf{h}_i + b_i).$$

The implementation computes all teacher-forced conditionals efficiently with a shifted cumulative sum. The shift is essential: prediction $i$ receives only pixels $x_{<i}$ and never the current or future pixels. The model is trained by minimizing the exact Bernoulli negative log-likelihood (NLL), summed over 784 pixels and averaged over images. It contains **201,616 trainable parameters**. Generation is

exact ancestral sampling: pixels are sampled one at a time and the hidden pre-activation is updated after every sampled pixel, requiring 784 sequential decisions per image.

## 4.2 Experimental setup

MNIST is loaded through the shared `data.py` used by Task 1.1. All 60,000 training images and 10,000 test images are deterministically binarized using `pixel > 0.5`, flattened to 784 dimensions in row-major order, and stored as `float32`. This keeps the likelihood comparison with FVSBN consistent.

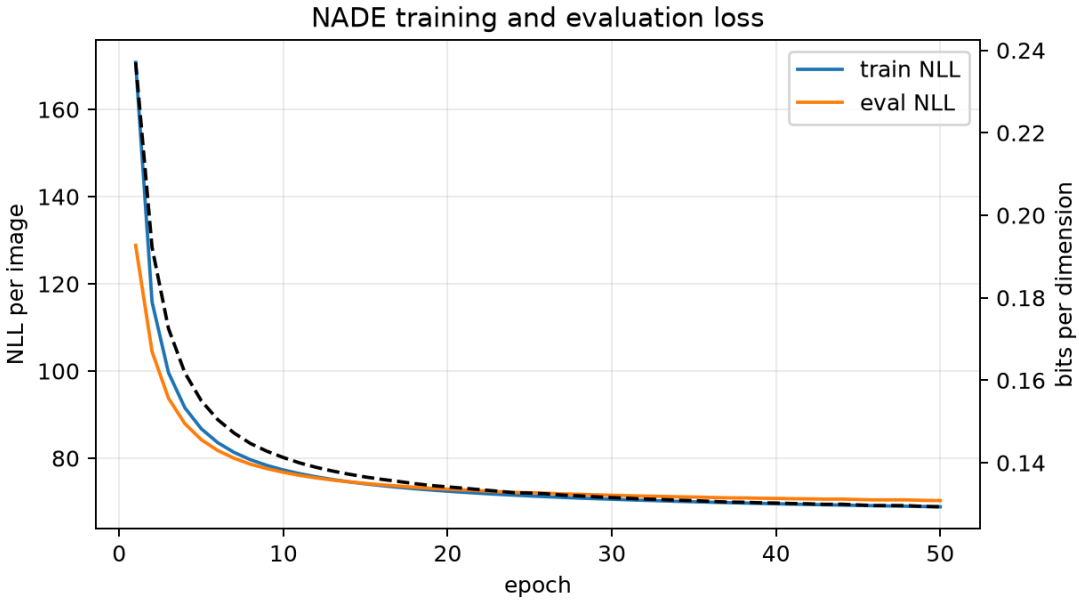
| Setting | Value |
|---|---|
| Hidden units | 128 |
| Epochs | 50 |
| Batch size | 64 |
| Optimizer | Adam |
| Learning rate | 0.001 |
| Random seed | 0 |
| Pixel order | Row-major |
| Hardware | NVIDIA GeForce RTX 3060 Laptop GPU |
| Software | PyTorch 2.14.0+cu130 |

Bits per dimension is calculated as $\mathrm{bpd} = \mathrm{NLL}/(784 \log 2)$. Sampling time is measured for 100 images after one warm-up, with CUDA synchronization and the median of five repetitions.

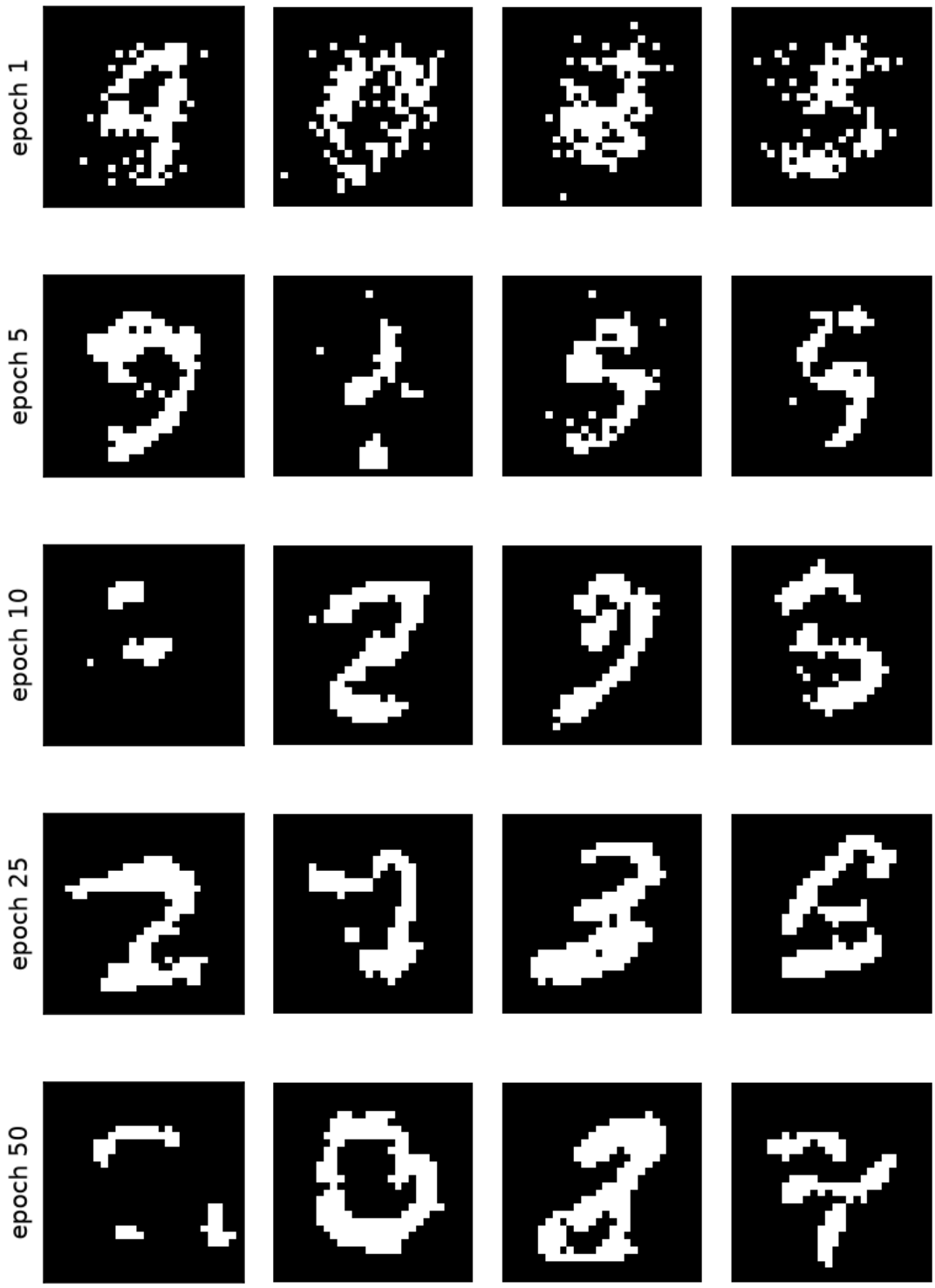
## 4.3 Results

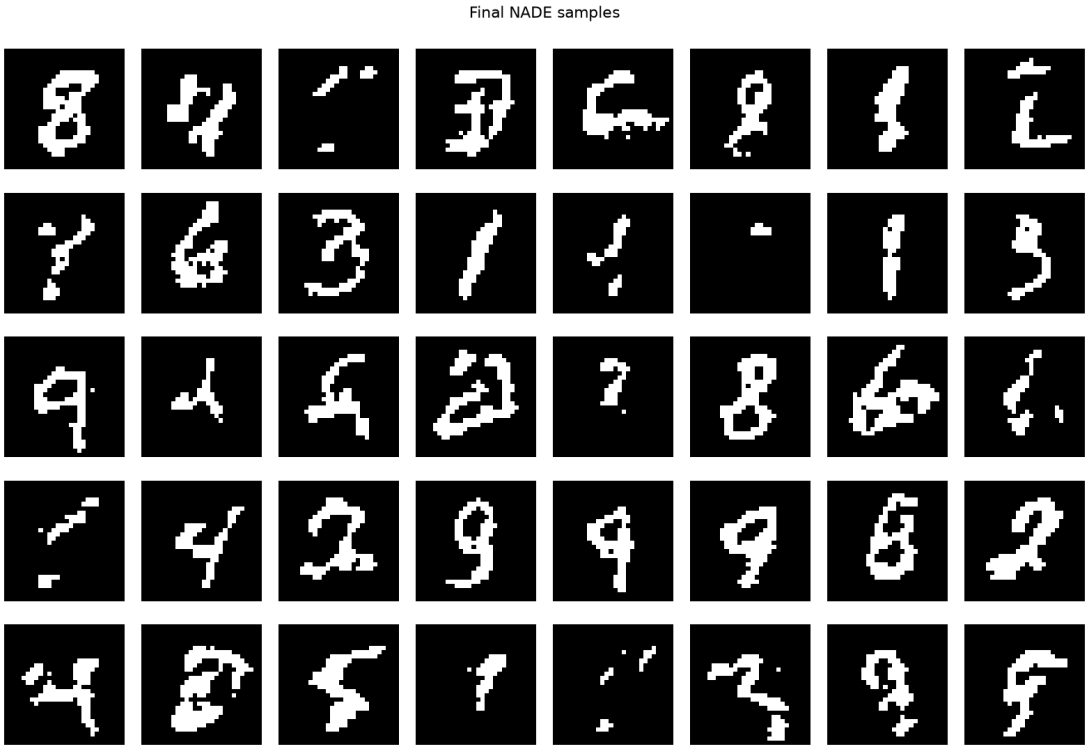
| Metric | Formal-run result |
|---|---|
| Final train NLL | 68.8562 nats/image |
| Test NLL | **70.2817 nats/image** |
| Test bits per dimension | **0.1293 bpd** |
| Train-test NLL gap | 1.4255 nats/image |
| Trainable parameters | **201,616** |
| Training time | **244.13 s** |
| Time for 100 samples | **0.1070 s** |

The model improved rapidly during the first ten epochs and then continued to improve more gradually. Test NLL decreased from 128.83 at epoch 1 to 84.24 at epoch 5, 76.77 at epoch 10, 72.09 at epoch 25, and 70.28 at epoch 50. The corresponding final test bpd was 0.1293. The train-test gap was only 1.43 nats, approximately 2% of the test NLL, so this run did not show severe overfitting.

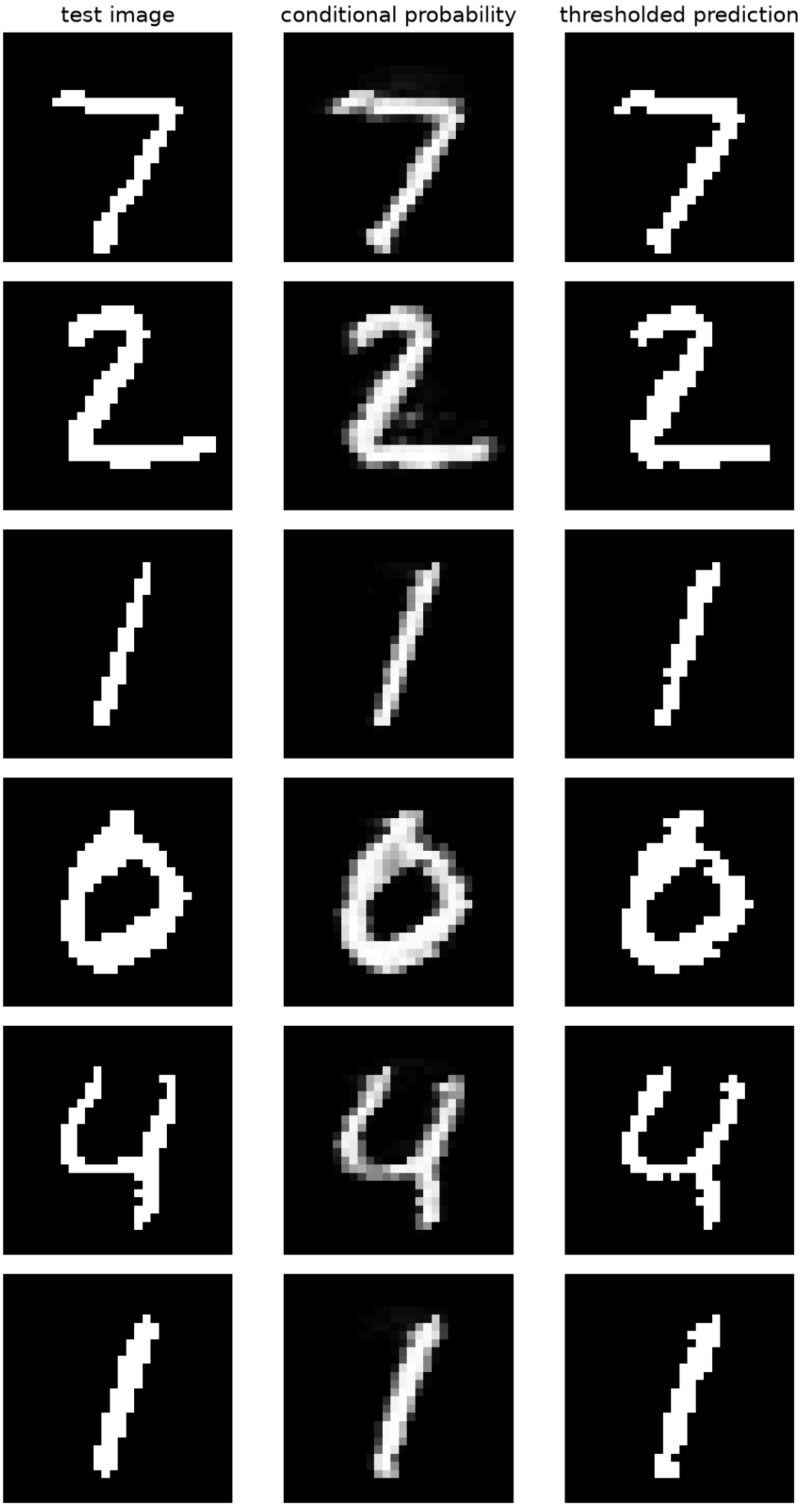



The staged samples show a clear qualitative progression. Epoch-1 samples contain substantial noise and disconnected pixels; recognizable digit structures begin to appear by epochs 5-10; and the epoch-25 and epoch-50 samples have more coherent strokes. The final sample grid contains several digit classes and writing styles, indicating useful diversity rather than obvious collapse. Most samples are recognizable, although some contain broken or overly thick strokes and a few are ambiguous between classes.


NADE samples during training
epoch 1
epoch 5
epoch 10
epoch 25
epoch 50




Teacher-forced probability maps reconstruct the test digits clearly, and their thresholded versions are close to the input images. They are cleaner than some free-running samples because each conditional receives the true prefix during teacher forcing, whereas an early sampling error becomes part of the context for all later pixels.


test image
conditional probability
thresholded prediction

## 4.4 Discussion

On the same binarized MNIST representation, NADE improved test NLL from the FVSBN result of 75.75 to 70.28 nats/image and improved bpd from 0.1394 to 0.1293, a reduction of approximately **7.2%**. NADE also used 201,616 parameters instead of 615,440, a reduction of approximately **67.2%**. These results support the value of the shared nonlinear hidden representation: it captures reusable dependencies among previous pixels more efficiently than independent linear conditional models.

The stronger likelihood does not make NADE cheaper in every respect. It required 244.13 s to train and 0.107 s to generate 100 images, whereas the available FVSBN records report 30-50 s and approximately 0.06 s. However, the FVSBN timing was collected on an RTX 3070 Laptop GPU and this NADE run used an RTX 3060 Laptop GPU, so the timing gap cannot be attributed solely to model architecture. Both autoregressive models also require **784 sequential pixel-generation steps**, not 28 steps.

## 4.5 Limitations

NADE generation is inherently sequential, making sampling slower than a parallel generator. Results can depend on the fixed row-major order, which does not explicitly preserve two-dimensional locality. Training uses true prefixes while generation uses previously sampled pixels, so errors can accumulate over the 784 steps. The model is unconditional and has no label control, and its fully connected structure does not exploit convolutional translation priors. Finally, the hidden-width ablation code is included but was not enabled in the formal run; therefore, no claim is made that 128 hidden units are optimal. A strict runtime comparison also requires rerunning all models on the same hardware and software stack.

## Task 1.2 comparison-table row

| Model | Test bpd | Parameters | Training time | 100 samples | Notes |
|---|---|---|---|---|---|
| NADE | **0.1293** | **201,616** | **244.1 s** | **0.107 s** | One hidden layer; shared nonlinear conditionals |